Lecture Notes/Summary for Unit 2: Program Correctness

These notes approximately summarize what was covered in lecture. They do not go into heavy detail, but go over the most important points which were brought up in lecture. These notes are not a replacement for going to lecture!

For a lot of the results, they are equivalent to result seen in the textbook, which usually provides them in more detail than these notes.

New! Some material in these notes was not brought up in lecture due to time constraints. This material is explicitly labeled with the "New!" tag you can see to the left.

Simple Recursive Program Correctness

Recall the definition of our set of trees $\mathcal{T}$:

Definition 1. *$\mathcal{T}$ is the smallest set satisfying:*

- $\varnothing \in \mathcal{T}$, *and*
- $\forall t_\ell, t_r \in \mathcal{T}. \forall n \in \mathbb{N}. Node(n, t_\ell, t_r) \in \mathcal{T}$.

Given this function, we can define the function $set : \mathcal{T} \to \mathcal{P}(\mathbb{N})$, which maps each tree to the set of natural numbers that appear in that tree.

$$\text{Example: } set\left(\begin{array}{ccc} & 5 & \\ / & & \backslash \\ \varnothing & & 3 \\ & / & \backslash \\ & \varnothing & \varnothing \end{array}\right) = \{3, 5\}$$

However, this function is difficult to evaluate, as its output depends on the global state of the tree. $set(t)$ relies on every part of t, and it's not immediately obvious how to compute it.

Instead, we can define an equivalent recursive function ts:

$$ts(t) = \begin{cases} \emptyset, & \text{if } t = \varnothing \\ \{n\} \cup ts(t_\ell) \cup ts(t_r), & \text{if } t = Node(n, t_\ell, t_r) \end{cases}$$

Note that $ts(t)$ is very direct to evaluate: It's purely local to a single node, and the amount of work per-node to be done is constant. However, it is not immediately obvious what $ts(t)$ is equal to for a more complicated t.

$set(t)$ is easy to interpret, but hard to compute. $ts(t)$ is easy to compute, but hard to interpret. Thus, to get the best of both worlds, it suffices to prove that $set(t) = ts(t)$.

We can do this using a proof by structural induction.

Proof. Proof omitted as an exercise. □

Thus, we see that induction and structural induction are very powerful tools for proving the correctness of recursive algorithms: Assuming that the function works on smaller inputs means that we only need to prove that the logic works for the non-recursive steps.

But what about an iterative program like the following?

1: **function** ArgMin(arr)

```
2:     r ← 1
3:     for i from 2 to n do
4:         if arr[i] < arr[r] then
5:             r ← i
6:         end if
7:     end for
8:     return r
9: end function
```

Here, we can't use any assumption on how ArgMin behaves on smaller inputs, and we have no way of deferring our logic to another proof. Instead, we need to prove the result about the entire loop at once. At the same time, given how many steps happen during the loop, it is infeasible to not break it down somehow. So how can we prove results about it?

Invariants

Let us start with an example.

Suppose there is some sequence of red and blue coins on the table.



You may repeatedly perform one of the following moves:

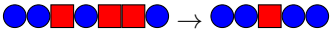
1. Insert two adjacent red coins anywhere in the sequence.



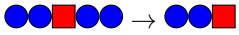
2. Insert two adjacent blue coins anywhere in the sequence.



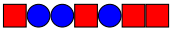
3. Remove two adjacent red coins from anywhere in the sequence.

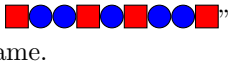


4. Remove two adjacent blue coins from anywhere in the sequence.



Our goal is then to start from some given starting sequence, and to then get to a target sequence. For example, suppose that we have the starting sequence and want to get to the ending sequence . We could then perform the following moves:

1. Insert two adjacent red coins at the start of the sequence: 
2. Insert two adjacent blue coins 3 steps into the sequence: 
3. Remove two adjacent red coins 1 step into the sequence: 
4. Insert two adjacent red coins at the end of the sequence: 
5. Insert two adjacent blue coins 6 steps into the sequence: 

Thus, we see that we can use these moves to get from  to  . We then say that the game "  →  " is possible. If no such sequence of moves existed, we would say that this is not a possible game.

But now, consider the game  →  . Is this game possible? Well, doing it in one move clearly isn't. What about two moves? What about 10? 1000?

The issue is that proving a possible requires just giving a series of moves. Proving a game not possible requires somehow proving that any series of moves does not work. This is a significantly more complicated process.

To prove this, we need to consider what would be true about any sequence of moves. And by looking at the above example sequence, we can observe the following fact: After adding and removing those coins, we are always left with an odd number of coins.

And if we can prove this, then we can actually prove that the game is impossible: If it were the case that after any number of moves we are left with an odd number of coins, then we cannot possibly reach the state  , as it would require us to reach an even number of coins.

This is an example of an invariant. An **invariant** is a property that is unchanged over time. Invariants are useful because they let us prove results that hold arbitrarily far into the future, which is very convenient for when we don't know exactly how long a process lasts.

Returning to our coin game example.

Program Correctness

The usefulness of invariants also extends to analysing practical programs. For example, consider the following function:

```
1: function ArgMin(arr)
2:     r ← 1
3:     for i from 2 to n do
4:         if arr[i] < arr[r] then
5:             r ← i
6:         end if
7:     end for
8:     return r
9: end function
```

This function finds the index of a minimum element in arr. But how can we prove this?

The trick is to use a **loop invariant**. A loop invariant is a special time of invariant, defined as holding at the end of each iteration of a given loop. In other words, it is a property that holds at the start and end of each iteration of a loop.

By looking at a few iterations of the loop when ArgMin runs, we can observe the following property: After a few iterations of the loop, r appears to be an index of the minimum in a prefix of the array.

More formally, we can observe the following loop invariant.

Lemma 1. *Let t be a number of iterations, and let $P(t)$ be defined as "after t iterations of the loop, r_t satisfies $\forall j \in [1..t+1].arr[r_t] \leq arr[j]$". Then $P(t)$ holds for all t.*

Here, we are using the notation r_t to mean "the value of r after t iterations of the loop". This will be our standard notation for this course.

Proof. To prove this, we will use induction. And more specifically, because the invariant depends on the number of iterations of the loop, we are doing induction over time: The property holding at one moment in time implies the property holding in the future.

To begin, we can prove $P(0)$. Note that the state "after 0 iterations of the loop" is the initial state of the program. Thus, $r_0 = 1$. And observe that 1 is trivially the index of the minimum of $arr[1..1]$, as that is the only index in the range.

Now, suppose that we knew that for some t, $P(t)$ held. In other words, $\forall j \in [1..t+1].arr[r_t] \leq arr[j]$. Consider the value of r after $t+1$ iterations. There are two possible cases, depending on if the if statement is true or not.

- **if $arr[i] < arr[r]$:** Then we update r_{t+1} to be equal to i_{t+1}, the value of i during iteration $t+1$. But note that as the loop starts at $i = 2$, that means that $r_{t+1} = t + 2$.

Then, note that the if statement guarantees that $arr[r_{t+1}] < arr[r_t]$. And furthermore, by our inductive hypothesis, $\forall j \in [1..t+1].arr[r_{t+1}] < arr[r_t] \leq arr[j]$. Furthermore, as $arr[r_{t+1}] = arr[t+2]$, that means that $P(t+1)$ holds.

- **if** $arr[i] \geq arr[r]$**:** Then r is not updated so $r_{t+1} = r_t$, and we also see that $arr[r_{t+1}] \leq arr[t+2]$. But that means that combined with our inductive hypothesis, $\forall j \in [1..t+2].arr[r_{t+1}] \leq arr[j]$, so $P(t+1)$.

Thus, by induction, $P(t)$ holds for all iterations t. □

This means that when the loop finishes, after iteration $n-1$ (where n is the length of the array), $\forall j \in [1..n].arr[r_{n-1}] \leq arr[j]$. And this value of r is returned, so we see that this returns the index of a minimum.

And thus, ArgMin is correct. ... Except, what does it mean for an algorithm to be correct? We've shown what ArgMin does, but we never actually specified what it is supposed to do.

To explain what a function does, we need to define its specifications, what it is supposed to do. We do this via preconditions and postconditions,

Definition 2. *A **precondition** is a requirement that must be true before an algorithm is run. This is necessary conditions for the algorithm to work correctly.*

*A **postcondition** is a guarantee that holds after an algorithm runs, given that all the preconditions for the algorithm held beforehand.*

In other words, preconditions are conditions that must be true before an algorithm is to be run. These are the requirements the algorithm writer sets that anyone using the algorithm must satisfy. If preconditions are not met, then an algorithm has undefined behaviour, and has no promises about what it will do.

Postconditions are conditions that must be true after an algorithm is run. These are promises the algorithm writer guarantees will hold if the algorithm is run while satisfying the preconditions. In other words, if an end user correctly runs the algorithm with all of its preconditions, then all of the postconditions are guaranteed to hold after the algorithm is finished running.

For our ArgMin example, we could have the following pre- and postconditions:

> Precondition: arr is an array of integers of length $n > 0$.
> Postcondition: The contents of arr are unchanged. ArgMin returns i such that $1 \leq i \leq n$ and such that $\forall j \in [n].arr[i] \leq arr[j]$.

Then, our look invariants above prove that ArgMin does actually obey its specification.

Insertion Sort

In lecture, we also saw InsertionSort. This has the following specifications and pseudocode:

> Precondition: arr is an array of integers of length $n > 0$.
> Postcondition: arr contains the same elements as it started with, but they are now sorted. In other words, $\forall 2 \leq j \leq n. \forall 1 \leq i \leq j. arr[i] \leq arr[j]$.

```
1:  function InsertionSort(arr)
2:      for i from 2 to arr.length do
3:          for j down from i − 1 to 1 do
4:              if arr[j + 1] < arr[j] then
5:                  swap arr[j + 1] and arr[j]
6:              else
7:                  break
8:              end if
9:          end for
10:     end for
11: end function
```

In lecture, we saw the following proof of its correctness. As this algorithm involved a nest for-loop, our proof had to involve nested loop invariants.

Lemma 2. *If the preconditions for* InsertionSort *holds, then the postconditions for* InsertionSort *will hold after it terminates.*

Proof. We will prove this via a loop invariant. More specifically, we will prove that after the first t iterations of the outer loop, $arr[1..t+1]$ is sorted.

However, note that as the array changes over time, we thus need to clear up our notation to make sure we are explicit about which version of arr we are talking about. Let arr_t be the value of arr after iteration t.

Then, we can define our loop invariant as follows: Let $P(t)$ be "after t iterations of the outer loop, $\forall 1 \leq k < \ell < t+1.arr_t[k] \leq arr_t[\ell]$, and arr_t contains the same elements as arr_0 does, potentially reordered". In other words, $arr_t[1..t+1]$ is sorted, and we don't create or delete any elements.

Observe that the second claim is trivial to prove by induction by observing that the only changes to arr come from swaps, which rearrange the elements, keeping them the same. As such, we will only prove the sorted claim.

First, observe that $P(0)$ holds as $arr[1..1]$ is a singleton array, which is trivially sorted.

Now, suppose that $P(t)$ held for some t, and consider iteration $t+1$. Note that as i starts at 2, iteration $t+1$ is thus equivalent to the following loop:

```
for j down from t + 1 to 1 do
    if arr[j + 1] < arr[j] then
        swap arr[j + 1] and arr[j]
    else
        break
    end if
end for
```

However, analysing an entire second loop is difficult in one step. After all, a lot of work might be done here. So instead, we will have a nested loop invariant! Note that as this entire inner loop happens during iteration $t+1$ of the outer loop, we can just work with that one value of $t+1$ for now.

For our given value of t, let $arr_{t,t'}$ be the contents of arr after iteration t' of the inner loop. Additionally, let $j_{t'}$ be the value of j during iteration t'. We can observe that $j_{t'} = t+2-t'$, and we will use this to define j_0. Then we claim that the following is an invariant for the inner loop: Let $Q(t')$ be "after t' iterations of the inner loop, the following two conditions hold:

- If $t' > 0$, then $arr_{t,t'}[j_{t'}] \leq arr_{t,t'}[j_{t'}+1]$, and
- The subarray $arr_{t,t'}[1..j_{t'}-1, j_{t'}+1..t+2]$ is sorted. In other words $arr_{t,t'}[1] \leq arr_{t,t'}[2] \leq \cdot \leq arr_{t,t'}[t+1-t'] \leq arr_{t,t'}[t+3-t'] \leq \cdots \leq arr_{t,t'}[t+2]$. In yet other words, $\forall k < \ell \in [t+2] \setminus \{t+2-t'\}.arr[k] \leq arr[\ell]$."

Lemma 3. *For all t', $Q(t')$ holds.*

Proof. We will now prove this via a nested proof by induction. Note that we know that $P(t)$ holds by the inductive hypothesis of our outer loop. This states that $\forall k < \ell \in [t+1].arr[k] \leq arr[\ell]$. Note that this is exactly the condition in $Q(0)$, so we know that $Q(0)$ holds.

Now, suppose that $Q(t')$ holds for some t', and consider iteration $t'+1$ of the inner loop.

Note first that either $arr_{t,t'}[j_{t'+1}+1] < arr_{t,t'}[j_{t'+1}]$ or not. In the latter case, observe that we do nothing, so $arr_{t,t'+1} = arr_{t,t'}$ and thus $arr_{t,t'}[j_{t'+1}] \leq arr_{t,t'}[j_{t'+1}+1]$ If that is true, however, then we swap $arr_{t,t'}[j_{t'+1}+1]$ and $arr_{t,t'}[j_{t'+1}]$. Thus, we get that

$$arr_{t,t'+1}[j_{t'+1}] = arr_{t,t'}[j_{t'+1}+1] < arr_{t,t'}[j_{t'+1}] = arr_{t,t'+1}[j_{t'+1}+1]$$

Thus, in either case, $arr_{t,t'+1}[j_{t'+1}] \leq arr_{t,t'+1}[j_{t'+1}+1]$, so the first part of the loop invariant holds.

Now, consider the second part. We know by $Q(t')$ holding that $\forall k < \ell \in [t+2] \setminus \{j_{t'}\}.arr[k] \leq arr[\ell]$. Now, we can consider two cases, depending on if $arr_{t,t'}[j_{t'+1}+1] < arr_{t,t'}[j_{t'+1}]$ or not.

- **If $arr_{t,t'}[j_{t'+1}+1] < arr_{t,t'}[j_{t'+1}]$:** Then we swap $arr_{t,t'}[j_{t'+1}]$ and $arr_{t,t'}[j_{t'+1}+1]$. But since $j_{t'+1} = j_{t'}-1$, that means that the following two sets are equal:

$$arr_{t,t'}[1..j_{t'}-1, j_{t'}+1..t+2] = arr_{t,t'+1}[1..j_{t'+1}-1, j_{t'+1}+1..t+2]$$

Less formally, by moving the out of place element down, it moves from index $j_{t'}$ to index $j_{t'+1}$, and thus the set disregarding that element remains sorted.

- **If** $arr_{t,t'}[j_{t'+1}+1] \geq arr_{t,t'}[j_{t'+1}]$**:** Then we break, and $arr_{t,t'+1} = arr_{t,t'}$. But by our inductive hypothesis, we know that $arr_{t,t'}[1..j_{t'}-1, j_{t'}+1..t+2] = arr_{t,t'+1}[1..j_{t'+1}, j_{t'+1}+2..t+2]$ is sorted. However, due to the case we're in, we know that $arr_{t,t'+1}[j_{t'+1}] \leq arr_{t,t'+1}[j_{t'+1}+1]$. At the same time, $arr_{t,t'+1}[j_{t'+1}+1] = arr_{t,t'}[j_{t'}] \leq arr_{t,t'}[j_{t'}+1] = arr_{t,t'+1}[j_{t'+1}+2]$ by $Q(t')$ holding (note that this does not apply if $t' = 0$, but if that is the case, this inequality is not relevant to what follows anyway).

  Thus, we see that $arr_{t,t'+1}[j_{t'+1}] \leq arr_{t,t'+1}[j_{t'+1}+1] \leq arr_{t,t'+1}[j_{t'+1}+2]$. And combined with the other result, we thus see that the entire subarray $arr_{t,t'+1}[1..t+2]$ is sorted.

Thus, $Q(t'+1)$ holds, and by induction, it holds for all t'. □

Now that we have our loop invariant, we can consider the outer loop again. Note that we can exit the loop in one of two ways: By reaching the end of the loop, or by hitting a break statement. The proof above just showed that if we hit a break statement, then $arr_{t+1}[1..t+2]$ is sorted. If we run to the end of the loop, however, then we still get that $arr_{t+1}[1..t+2]$ is sorted as we finish with iteration $t' = t+1$, and so our loop invariant gives that $arr_{t+1}[1] \leq arr_{t+1}[2]$ and that $arr_{t+1}[2..t+2]$ is sorted. This exactly proves that $arr_{t+1}[1..t+2]$ is sorted.

Thus, we see that $P(t+1)$ holds in all cases. Thus, by induction, $P(t)$ holds for all t.

The last iteration of the outer loop is with $t = n-1$, as the loop runs from 2 to n. Thus, at the end of the last loop, $arr_{n-1}[1..n]$ is sorted. But this means the entire array is sorted, and we are done. □

Correctness and Termination

Let us look at one more example. Consider binary search. It has the following specification and its implementation:

PRECONDITION: arr is a sorted array of integers of length $n > 0$. x is an integer.
POSTCONDITION: arr is unchanged. BINSEARCH returns a natural number i such that

- $0 \leq i \leq n$
- If $\exists j \in [n].arr[j] = x$, $i \geq 1$ and $arr[i] = x$.
- If not, $i = 0$.

```
1:  function BinSearch(arr, x)
2:      start ← 1;      end ← arr.length
3:      while start < end do
4:          mid ← ⌊(start+end)/2⌋
5:          if arr[mid] ≥ x then
6:              end ← mid
7:          else
8:              start ← mid + 1
9:          end if
10:     end while
11:     if arr[start] = x then
12:         return start
13:     else
14:         return 0
15:     end if
16: end function
```

Let us attempt to prove the correctness of this algorithm. Once again, we need to define a loop invariant:

Lemma 4. *For any number t of loop iterations, the following loop invariant $P(t)$ holds:*

- $1 \leq start_t \leq end_t \leq n$, *and*

- *if* $\exists j \in [n].arr[j] = x$*, then* $\exists i \in [start_t..end_t].arr[i] = x$.

Proof. As always, let us use a proof by induction. First, note that $start_0 = 1$ and $end_0 = n$. Thus, the first part of $P(0)$ holds trivially. But simultaneously, that also means that $[start_0..end_0] = [n]$, so the second part of $P(0)$ becomes a tautology. Thus, $P(0)$ holds.

Now, suppose that for some t, $P(t)$ holds. Consider the $(t+1)$st iteration of the loop. Let us prove the two parts of $P(t+1)$ separately.

First, observe that we know $1 \leq start_t \leq end_t \leq n$ from $P(t)$. But note that it iteration $t+1$ occurs, that means that the loop condition must still be true at the start of that iteration. This thus tells us that $start_t < end_t$ must be true, or else iteration $t+1$ would never have happened.

Now, consider mid_{t+1}. As $mid_{t+1} = \lfloor \frac{start_t + end_t}{2} \rfloor$ and as $start_t \leq end_t$, we can conclude that $start_t \leq mid_{t+1} < end_t$. We can then consider the two cases for what happens during iteration $t+1$:

- **If** $arr[mid_{t+1}] \geq x$**:** In this case, we update $end \leftarrow mid$. This means that $end_{t+1} = mid_{t+1}$ and $start_{t+1} = start_t$. But then $1 \leq start_{t+1} = start_t \leq mid_{t+1} \leq end_{t+1} \leq n$, so the first part of $P(t+1)$ holds.

- **If** $arr[mid_{t+1}] < x$**:** In this case, we update $start \leftarrow mid + 1$. This means that $start_{t+1} = mid_{t+1} + 1$ and $end_{t+1} = end_t$. But then $1 \leq start_t \leq mid_{t+1} + 1 = start_{t+1} \leq end_t = end_{t+1} \leq n$, so the first part of $P(t+1)$ holds.

Thus, in both cases, $1 \leq start_{t+1} \leq end_{t+1} \leq n$.

Now, let us consider the second part of $P(t+1)$. As this is an implication that we want to prove, we will begin by assuming that $\exists j \in [n].arr[j] = x$. Then, by our inductive hypothesis, we know that $\exists i \in [start_t..end_t].arr[i] = x$. In other words, there must be some index in our range which contains x.

Now, we can once again consider our cases:

- **If** $arr[mid_{t+1}] < x$**:** Note that as arr is sorted by our preconditions, that means that $arr[start_t] \leq arr[start_t + 1] \leq \cdots \leq arr[mid_{t+1}] < x$. This means that $i > mid_{t+1}$, which implies that $i \in [mid_{t+1} + 1..end_t]$. As $end_{t+1} = end_t$ and $start_{t+1} = mid_{t+1} + 1$ in this case, that thus proves that $\exists i \in [start_{t+1}..end_{t+1}].arr[i] = x$. Thus, the second part of $P(t+1)$ holds.

- **If** $arr[mid_{t+1}] > x$**:** This proceeds analogously to the above, but with the inequalities flipped.

- **If** $arr[mid_{t+1}] = x$**:** If $arr[mid] = x$, then we update $end_{t+1} = mid_{t+1}$ and $start_{t+1} = start_t$. Thus, $mid_{t+1} \in [start_{t+1}..end_{t+1}]$ and $arr[mid_{t+1}] = x$, so the second part of $P(t+1)$ holds.

Thus, in all cases, $P(t+1)$ holds, so by induction $P(t)$ holds for all t. □

As such, we see that our loop invariant holds.

However, we are not yet done proving the correctness of this algorithm. After all, there is extra code left after the loop, which we now have to consider.

After the loop ends, we know that the loop invariant holds with the final values of the variables. At the same time, because the loop ended, that means that the loop condition was false with the last values of *start* and *end*. So if the last iteration was t', then the loop invariant indicates that $1 \leq start_{t'} \leq end_{t'} \leq n$ and the loop condition being false indicates that $start_{t'} \geq end_{t'}$. Thus, we can derive that $start_{t'} = end_{t'}$.

Now, we can consider two cases:

- There is an x in arr: If so, then we know by our loop invariant that $\exists i \in [start_{t'}..end_{t'}].arr[i] = x$. However, $start_{t'} = end_{t'}$, so that implies that $arr[start_{t'}] = x$. But then the next if statement would be true, so the algorithm returns $start_{t'}$. Furthermore, the loop invariant indicates that $1 \leq start_{t'} \leq n$. Thus, the postconditions are satisfied.

- There is no x in arr: In that case, $arr[start_{t'}] \neq x$, by definition. Thus, we return 0, so the postconditions are satisfied.

And thus, we prove that when the algorithm finishes, we satisfy all the postconditions. Thus, we know the program is correct.

... Well, almost. We are missing one small element. We said "when the algorithm finishes". But we should hav esaid "if the algorithm finishes". See, we actually don't know for a fact that the algorithm ever stops running,

This gets to a more general comment about notions of correctness. So far, in all our examples, we have shown that if our preconditions hold, then the postconditions must hold after the algorithm terminates. But proving that the algorithm terminates is itself a claim that needs to be proven. As such, we add several new definitions.

- **Partial Correctness**: If the preconditions of the program hold, and the program halts, then the postconditions will hold after it terminates.
- **Termination**: If the preconditions of the program hold, then the program will halt after a finite amount of time.
- **Correctness**: Partial Correctness and Termination. In other words, if the preconditions of the program hold, then the program halts after a finite amount of time and the postconditions will hold after it terminates.

To prove an algorithm correct, we must prove both partial correctness and termination. So far, all our algorithms trivially terminated as each loop would only repeat a finite number of times by definition. But with this last algorithm, it becomes nontrivial.

To justify it, we observe that the "range" we are considering shrinks each time. Specifically, consider $end_t - start_t$. Observe that by our loop invariant, this is a non-negative number. Furthermore, note that as $start_t \leq mid_{t+1} \leq end_t$, either $end_{t+1} - start_{t+1} = end_t - (mid_{t+1} + 1)$ or $end_{t+1} - start_{t+1} = mid_{t+1} - start_{t+1}$. In either case, we see that $end_{t+1} - start_{t+1} < end_t - start_t$. Thus, this sequence is a decreasing sequence of natural numbers. This is finite (see problem set 1, using the Well-Ordering Principle), and thus we must have a finite number of iterations. And thus, the algorithm must terminate.

Thus, we have fully proven the correctness of binary search, as desired.